

Détection de perte de précision dans le cadre de l'analyse statique d'un fichier binaire exécutable

Bastien Padeloup

07/09/2012

Sommaire

- 1 Introduction
- 2 Techniques d'analyse
- 3 Travail effectué
- 4 Conclusion

Sommaire

- 1 Introduction
- 2 Techniques d'analyse
- 3 Travail effectué
- 4 Conclusion

Analyse statique

Définition

- Analyse du code (source ou désassemblé) d'un programme
- Aucune exécution
- *Model-checking*, reconstruction de structures, ...

Pourquoi l'analyse statique ?

- Contexte de virologie informatique
- Risques de fuites
- Détection de virus métamorphiques
- Plus exhaustif que l'analyse dynamique
- Application de techniques (VSA, ...)

Analyse de fichiers binaires

Avantages

- Abstraction des optimisations du compilateur
- Virus : altérations post-compilation

Limites

- Instructions en assembleur ayant des effets de bord
- Nécessité de définir une représentation intermédiaire

Problème

Besoins

- Fichier désassemblé
- Outil d'analyse

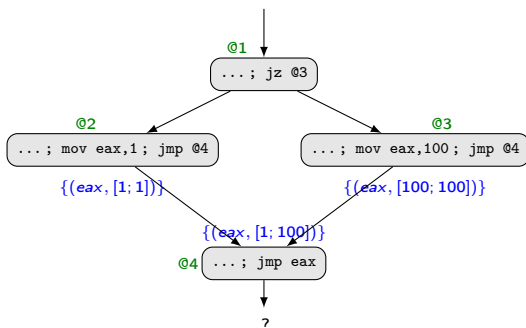
Difficulté

- Désassembler un fichier binaire est indécidable
- Nécessité de suivre le flot de contrôle
- Instructions du type `<< jmp eax >>`

Exemple

Structure d'indirection

- Très mauvaise analyse
- Détecter des pertes de précision



Sommaire

- 1 Introduction
- 2 Techniques d'analyse
- 3 Travail effectué
- 4 Conclusion

Technique d'analyse dynamique

Idée

- Simuler l'exécution en machine virtuelle

Avantage

- Précision du résultat

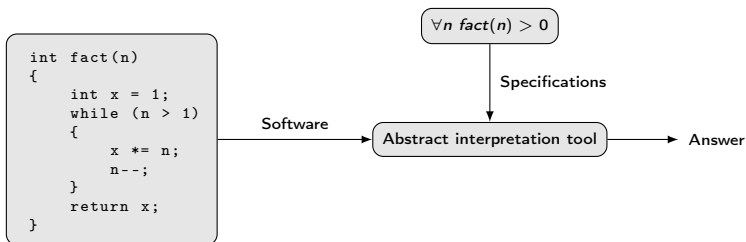
Inconvénients

- Test exhaustif souvent impossible
- Incomplétude du résultat
- Terminaison pas assurée

Technique d'analyse statique

Idée

- Prouver des propriétés sur le programme
- Sans exécuter le code



Technique d'analyse statique

Avantages

- Connaissance de toutes les valeurs possibles pour toutes les variables à toutes les instructions
- Technique complète

Inconvénients

- Temps de convergence long, parfois infini

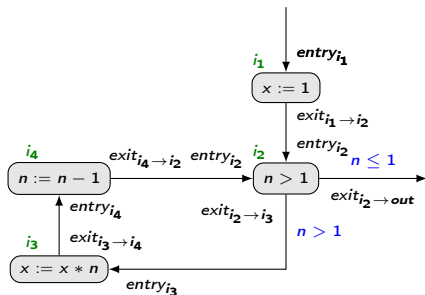
Limitations

- Représentation convexe des ensembles
- Perte volontaire de précision

Equations représentant la dynamique du programme

Objectif

- Représenter la dynamique du programme par des équations
- Résoudre le système d'équations (équation de point fixe)



$$\vec{P} = \begin{cases} \text{entry}_{i_1} = \{(n, \top); (x, \top)\} \\ \text{exit}_{i_1 \rightarrow i_2} = \text{entry}_{i_1} \setminus (x, \bullet) \cup (x, 1) \\ \text{entry}_{i_2} = \text{exit}_{i_1 \rightarrow i_2} \cup \text{exit}_{i_4 \rightarrow i_2} \\ \text{exit}_{i_2 \rightarrow i_3} = \text{entry}_{i_2} \setminus (n, \nu) \cup (n, \nu \cap [2; +\infty]) \\ \text{exit}_{i_2 \rightarrow \text{end}} = \text{entry}_{i_2} \setminus (n, \nu) \cup (n, \nu \cap]-\infty; 1]) \\ \text{entry}_{i_3} = \text{exit}_{i_2 \rightarrow i_3} \\ \text{exit}_{i_3 \rightarrow i_4} = \text{entry}_{i_3} \setminus (x, \nu) \cup (x, \nu * n) \\ \text{entry}_{i_4} = \text{exit}_{i_3 \rightarrow i_4} \\ \text{exit}_{i_4 \rightarrow i_2} = \text{entry}_{i_4} \setminus (n, \nu) \cup (n, \nu - 1) \end{cases}$$

Sommaire

- 1 Introduction
- 2 Techniques d'analyse
- 3 Travail effectué**
- 4 Conclusion

Rappel

Objectif

- Caractériser précisément `eax` dans `<< jmp eax >>`

Problème

- Perte de précision induite par l'interprétation abstraite
- Une valeur en trop entraîne une branche en trop à explorer
- Une *fausse* branche peut contenir du *vrai* code (indécidable)

Conséquences de la perte de précision

- Détection de malware : faux positifs
- Désassemblage : analyse impossible

Présentation

Objectif

- Pour une structure donnée, caractériser la perte de précision
- Lancer ou non une correction

Idée

- Compter les configurations possibles

Proposition

- Si une structure s a k configurations possibles de n variables manipulées par s , alors il est possible de majorer le nombre de configurations en sortie de s par k
- Notation : $sup(s) = k$

Exemple

Instruction élémentaire

- $x := \text{mul}(y, z)$

Extensions

- Instructions
- Blocs de base

Autres indicateurs (heuristiques)

Borne inférieure

- Fuzzing sur k configurations parmi n possibles
- Notation : $\text{inf}(s)$

Tendance

- Rapport $\frac{\text{inf}(s)}{k} * n$
- Emplacement dans $[\text{inf}(s); \text{sup}(s)]$
- Notation : $\bar{s}$

Différents niveaux d'analyse

Besoin

- Mettre en avant les structures de code entraînant le plus de perte de précision

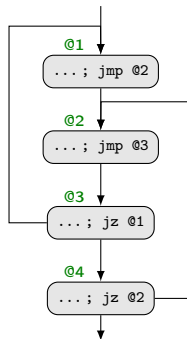
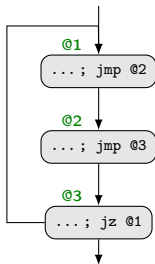
Structures existantes

- Instruction élémentaire, instruction, bloc de base
- Pas de dynamique du programme

Structure de boucle

Problème

- Pas de structure *propre* de boucle en assembleur
- Possibilité de démarrer une boucle au milieu d'une autre



Structure de boucle

Choix

- Détecter les composantes fortement connexes (CFC) du CFG
- Algorithmes : Tarjan, Kosaraju-Sharir, ...

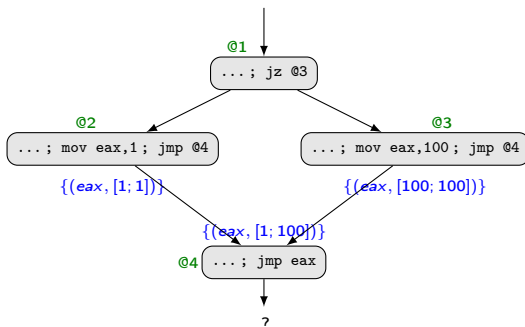
Propriétés

- Un graphe sous forme de CFC est un DAG
- Autant de configurations en sortie qu'en entrée

Structure d'indirection

Problème

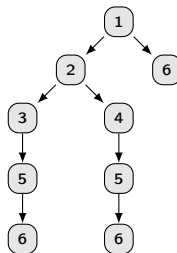
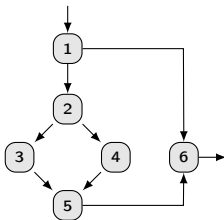
- Entrée d'un bloc par union des prédécesseur
- Grande perte de précision si représentation convexe



Structure d'indirection

Idée

- Déplier le graphe sous forme de CFC
- Pour tout nœud n ayant plusieurs fils, chercher le premier descendant commun n'
- (n, N, n') forme une indirection (N : nœuds parcourus)



Séquence de structures

Idée

- S'inspirer des blocs de base

Structures disponibles

- Instruction élémentaire
- Instruction
- Bloc de base
- Boucle
- Indirection
- Séquence

Idées

Paramètres intéressants

- Type de structure
- Emplacement dans le programme
- Appartenante à une trace d'exécution
- Ecart entre les configurations trouvées et $\bar{s}_j$
- Dépassement de $sup(s)$
- ...

Résolution

Nombre de n -uplets fini en entrée

- Evaluation de code
- Conservation de la complétude
- Changement de treillis si nécessaire
- Attention à la terminaison

Nombre de n -uplets infini en entrée

- Collecte d'informations (courbes de tendance, ...)
- Méthodes stochastiques (échantillonnage, extrapolation, ...)
- Intervention de l'analyste

Sommaire

- 1 Introduction
- 2 Techniques d'analyse
- 3 Travail effectué
- 4 Conclusion

Contexte

Objectif

- Value sets analysis
- Nécessité d'avoir un désassemblage complet

Problèmes

- Sauts à destination variable
- Perte de précision par l'interprétation abstraite

Travaux

Technique mixte

- Interprétation abstraite et fuzzing
- Détection des pertes de précision

Choix du raffinement

- Estimateurs de la perte de précision
- Découpage en structures

Autres domaines étudiés

- Langage intermédiaire
- Détermination de l'état des flags
- Accélération de la convergence

Extensions possibles

Théorie

- Recherche de sous-structures dans les boucles
- Extension aux domaines relationnels

Pratique

- Implémentation (POC en Java)
- Tester et développer des heuristiques efficaces

